

一、实验概述

本项目涵盖了数字 IC 设计的三大核心环节：**前端 RTL 设计**、**逻辑综合**和**版图物理设计**。以一款支持 MiniRV 指令集（37 条指令）的单周期 RISC-V 处理器为目标，基于 SMIC 0.13μm 工艺，使用 Synopsys 全套 EDA 工具链完成了从 RTL 到 GDSII 前的全流程设计。

三个实验的关系反映了数字芯片设计的真实工业流程——先有 RTL 设计并验证功能正确，再通过逻辑综合将 RTL 映射为门级网表，最后通过物理设计完成布局布线并提取实际延迟：

实验一：前端设计
(RTL)

实验二：逻辑综合
(门级网表)

实验三：版图设计
(版图 + SPEF + SDF)

SystemVerilog
Vivado

Design Compiler
DC 2020.09

ICC2 + PrimeTime + VCS
ICC2 2022.03 / PT 2022.03

实验一（前端）通过 5 个子实验递进完成：ALU → 数据通路 → 存储器扩展 → 控制器 → MiniRV CPU。实验二在实验一的基础上进行逻辑综合，添加 IO PAD 并生成门级网表。实验三对实验二的网表进行布局布线，提取寄生参数，完成后仿真。

二、实验环境

项目	详情
设计语言	SystemVerilog / Verilog
前端仿真	Vivado Simulator (XSim), Verilator 5.020
逻辑综合	Synopsys Design Compiler R-2020.09-SP4
版图设计	Synopsys IC Compiler II (ICC2) T-2022.03
时序签核	Synopsys PrimeTime (PT) T-2022.03
门级仿真	Synopsys VCS T-2022.06
工艺	SMIC 0.13μm, 1P8M (1 层 Poly + 8 层金属)
工作条件	typical, 1.2V, 25°C
标准单元库	typical_1v2c25.db / smic13g (MW 物理库)
IO PAD 库	SP013D3_V1p2_typ.db / SP013D3_V1p2_8MT (MW)
符号库	smic13g.sdb
寄生模型	StarRC TLUPLUS (TM9k_MIM1f, p1mt8)

三、实验一：前端 RTL 设计

3.1 设计目标

从零开始实现一款 32 位单周期 RISC-V 处理器。先从 ALU 等底层部件开始，逐步构建完整的数据通路和控制逻辑，最终实现支持 37 条指令的 MiniRV CPU。通过差分测试框架验证功能正确性。

3.2 设计过程

前端设计通过 5 个递进式子实验完成，每个子实验在前一个基础上增加新的功能：

子实验 1：ALU 设计

实现参数化的 32 位算术逻辑单元，支持 4 种运算操作：

ALUControl	功能	用途
00	Add	加法（算术运算 + 地址计算）
01	Sub	减法（算术运算 + 比较）
10	And	按位与
11	Or	按位或

同时计算 4 个状态标志位：

标志	含义	计算方式
N (Negative)	负标志	结果最高位 <code>Result[31]</code>
Z (Zero)	零标志	结果全零则为 1
C (Carry)	进位/借位	加法：扩展加法进位；减法： <code>A >= B</code> （无符号比较）
V (Overflow)	有符号溢出	加法：同号操作数、异号结果；减法：异号操作数、结果与 A 异号

使用 `always_comb` 组合逻辑实现。加法使用扩展位 `{1'b0, A} + {1'b0, B}` 产生进位输出。通过穷举测试验证：4 种运算 × 256 种 A × 256 种 B = 262,144 组测试向量全部通过。

子实验 2：数据通路

完成 CPU 数据通路中全部核心部件，共 8 个模块（含 ALU）：

模块	类型	关键特性
PC	时序逻辑（异步复位）	<code>always_ff @(posedge clk, posedge rst)</code> , 复位清零
Adder	组合逻辑	用于 PC+4 和跳转地址计算
MUX	组合逻辑	参数化位宽的二选一选择器
RegFile	混合（组合读 ×2 + 时序写 ×1）	32×32bit, x0 硬连线为 0
ImmGen	组合逻辑	支持 I/S/B 三种立即数格式的提取和符号扩展
IM	混合（组合读）	字节地址小端序组装 32 位指令字

模块	类型	关键特性
DM	混合（组合读 + 时序写）	异步复位，字节级读写

子实验 3：存储器扩展

从 256×8bit 基础 RAM 单元出发，通过层次化设计构建 1024×32bit 存储器：

```
mini_ram (256×8bit)
  ↓ ×4 片，字扩展（2-4 译码器 → CS 片选）
mini_ram_wortext (1024×8bit)
  ↓ ×4 片，位扩展（并行字节拆分/拼接）
mini_ram_bitext (1024×32bit)
```

关键技术：`generate for` 循环例化 + 片选译码 + 字节拆分 (`ram_data_i[i*8 +: 8]`)。

子实验 4：控制器与 CPU 集成

实现两级译码控制器：

```
instr[6:0]  → Control  → ALUOP[1:0], RegWrite, ALUSrc, MemWrite,
MemToReg, Branch
instr[14:12], instr[30] → ALU_controller → ALUControl[1:0]
```

指令	opcode	ALUSrc	MemToReg	RegWrite	MemWrite	Branch	ALUOP
R-type	0110011	0	0	1	0	0	10
lw	0000011	1	1	1	0	0	00
sw	0100011	1	X	0	1	0	00
beq	1100011	0	X	0	0	1	01

将 13 个模块 + 3 个 MUX 按照架构图连接为完整单周期 CPU。关键控制：**PcSrc = Branch & Zero**。仿真执行 10 条指令序列（lw ×3, sub, and ×2, sw ×2, or, beq），所有断言通过。

子实验 5：MiniRV CPU 验证

将 7 指令核心 CPU 升级为支持 37 条指令。主要架构变更：

NPC 重设计——从 2 种跳转模式扩展到 4 种：

NpcOp	npc 计算	对应指令
00	pc + 4	顺序执行
01	isTrue ? pc+offset : pc+4	条件分支
10	offset & ~1	jalr

NpcOp	npc 计算	对应指令
11	pc + offset	jal

译码方式——从分级译码（Control → ALUOP + ALU_controller → ALUControl）改为单级译码（ACTL 直接 opcode+funct → 4-bit ALUControl）。

控制信号扩展——新增 ALUSrcA（auipc 选择 PC 作为 ALU A 输入）、OffsetOrigin（jalr 选择 ALU 结果作为 NPC offset）、MemToReg 扩展为 2-bit（增加 IMM 和 PC+4 来源）。

ALU 扩展——从 4 种操作扩展到 14 种，每种比较操作同时输出 Result 和 isTrue。

字节/半字访存——Load 端根据 funct3 和地址对齐提取字节/半字并做符号/零扩展；Store 端读-改-写合并（从 DRAM 读出旧字，替换对应字节后写回）。

3.3 MiniRV 指令集（37 条）

类别	指令	数量	opcode
R-type	add, sub, and, or, xor, sll, srl, sra, slt, sltu	10	0110011
I-type ALU	addi, andi, ori, xori, slli, srli, srai, slti, sltiu	9	0010011
Load	lb, lbu, lh, lhu, lw	5	0000011
Store	sb, sh, sw	3	0100011
Branch	beq, bne, blt, bltu, bge, bgeu	6	1100011
U-type	lui, auipc	2	0110111 / 0010111
J-type	jal, jalr	2	1101111 / 1100111

3.4 验证方法：差分测试

cdp-tests 框架使用 Verilator 将设计编译为 C++ 仿真模型，与 C 语言 golden_model 执行同一指令，逐周期比对 5 个 debug 信号：

信号	说明
debug_wb_have_inst	当前周期是否有指令写回
debug_wb_pc	写回指令的 PC
debug_wb_ena	寄存器写使能
debug_wb_reg	写入的目标寄存器号
debug_wb_value	写入的寄存器值

调试中修复了两个关键问题：

- Debug 时序**：组合逻辑 debug 信号在时钟沿后显示下一条指令的结果，需改为 `always_ff @(posedge cpu_clk)` 寄存器化，在时钟上升沿捕获当前指令的执行结果。

- **字节/半字访存**：lb/lbu/lh/luh 需要从 32-bit 字中提取对应字节/半字并扩展；sb/sh 需要读-改-写合并（从 DRAM 读出旧值，替换后写回）。

最终 37/37 全部测试通过：

```

✓ add addi and andi auipc beq bge bgeu blt bltu bne
✓ jal jalr lb lbu lh lhu lui lw or ori sb sh
✓ sll slli slt slti sltiu sltu sra srai srl srli sub sw xor xori

```

3.5 结果截图



```

fupeiyi@user-Super-Server: /mnt/home/fupeiyi/proj/cpu-expr-new/cdp-tests$ python3 run_all_tests.py
[mycpu] Resetting ...
[INFO] Instruction ROM initialized with meminit.bin
[INFO] Data RAM initialized with meminit.bin
[mycpu] Reset done.
[diffptest] Test Start!
ECALL at PC = 0x000004e8, Stop now.
Test Point Pass!
===== xor END =====

===== SUMMARY =====
Passed Tests:
lh, beq, sh, sltiu, lbu, bne, and, sw, srl, start, lb, slti, simple, auipc, sll, or, bltu, lhu, jalr, add, bgeu, sltu, lw, blt, slt, lui, sra
, slli, srai, bge, addi, sb, andi, jal, ori, sub, srli, xori, xor
Failed Tests:

```

四、实验二：逻辑综合

4.1 设计目标

将实验一完成的前端 RTL 通过 Design Compiler 综合为 SMIC 0.13μm 门级网表，添加 IO PAD 作为芯片顶层，生成 SDC 时序约束和 SDF 延迟文件，为版图设计提供输入。

4.2 综合环境搭建

编写 `.synopsys_dc.setup` + `common_setup.tcl` + `dc_setup.tcl` 三层配置文件：

```

# common_setup.tcl 中的库设置
set TARGET_LIBRARY_FILES "\
/home/eda/lib/smic/aci/sc-x/synopsys/typical_1v2c25.db \
/home/eda/lib/smic/SP013D3_V1p4/syn/SP013D3_V1p2_typ.db"

set target_library $TARGET_LIBRARY_FILES
set link_library "*" $TARGET_LIBRARY_FILES
set symbol_library $SYMBOL_LIBRARY_FILES

```

`target_library` 指定 DC 映射目标的标准单元库和 IO PAD 库。`link_library` 中的 `*` 表示 DesignWare 基础组件库（加法器、乘法器等）。DC 自动加载 `.synopsys_dc.setup`，无需手动 source。

4.3 PAD 顶层设计

编写 `cpu_pad.sv`，在 myCPU 包裹 IO PAD 单元。PAD 是芯片内部逻辑与外部引脚的接口，提供 ESD 保护、电平转换和驱动能力。

处理了 myCPU 的全部 ~230 个 IO 端口：

信号	方向	位宽	PAD 类型	功能
rst_n	input	1	PI	复位（低有效 → 内部高有效）
clk	input	1	PI	系统时钟
irom_data	input	32	PI × 32	指令存储器数据输入
dram_rdata	input	32	PI × 32	数据存储器数据输入
irom_addr	output	32	PO8 × 32	指令地址输出
dram_addr	output	32	PO8 × 32	数据地址输出
dram_wdata	output	32	PO8 × 32	数据写入
dram_wen	output	1	PO8	写使能
debug 信号	output	71	PO8 × 71	调试接口

宽总线使用 `generate for` 批量例化。综合时所有 PAD 设为 `dont_touch`，DC 不会移除或优化。

4.4 约束设置

```
# 时钟：50MHz
create_clock -period 20.0 -name main_clk [get_ports clk_pad]
set_clock_uncertainty 0.2 [get_clocks main_clk]

# 输入约束
set_driving_cell -library typical_1v2c25 -lib_cell AND2X4
[all_inputs_no_clk_rst]
set_input_delay 0.1 -max -clock main_clk [all_inputs_no_clk_rst]

# 输出约束
set_output_delay 1.0 -max -clock main_clk [all_outputs]
set_load [expr [load_of typical_1v2c25/AND2X4/A] * 15] [all_outputs]

# 综合
compile_ultra
```

- `set_clock_uncertainty 0.2`：为时钟抖动和偏差预留 200ps 余量
- `set_driving_cell`：假设输入由库中最小的 AND2X4 驱动，保守估计输入 transition
- `set_input_delay 0.1`：前级外部延迟 100ps
- `set_output_delay 1.0`：后级外部路径需要 1ns
- `set_load ×15`：输出负载等效于 15 个标准输入电容

4.5 综合结果（50MHz @ SMIC 0.13μm）

指标	数值	说明
Slack	0.00ns	刚好满足 20ns 周期
关键路径长度	18.70ns	其中 PAD 贡献约 7.8ns 输入延迟
逻辑级数	47 级	32 位 ALU + 多路选择的典型深度
标准单元面积	73,175 μm²	组合 37,925 + 非组合 35,250
PAD 面积	1,613,430 μm²	234 个 PAD × ~6,900 μm²/个
总单元面积	1,686,605 μm²	PAD 占 96%，是 PAD-limited 设计
等效门数	~5,600	(不含 PAD)
总单元数	5,556	组合 4,229 + 时序 1,093
动态功耗	~252 mW	单元内部 179mW + 连线开关 ~73mW

1,093 个时序单元对应 32×32=1,024 bit 寄存器堆 + 32 bit PC + 32 bit debug 寄存器 ≈ 1,088 bit，与预期吻合。

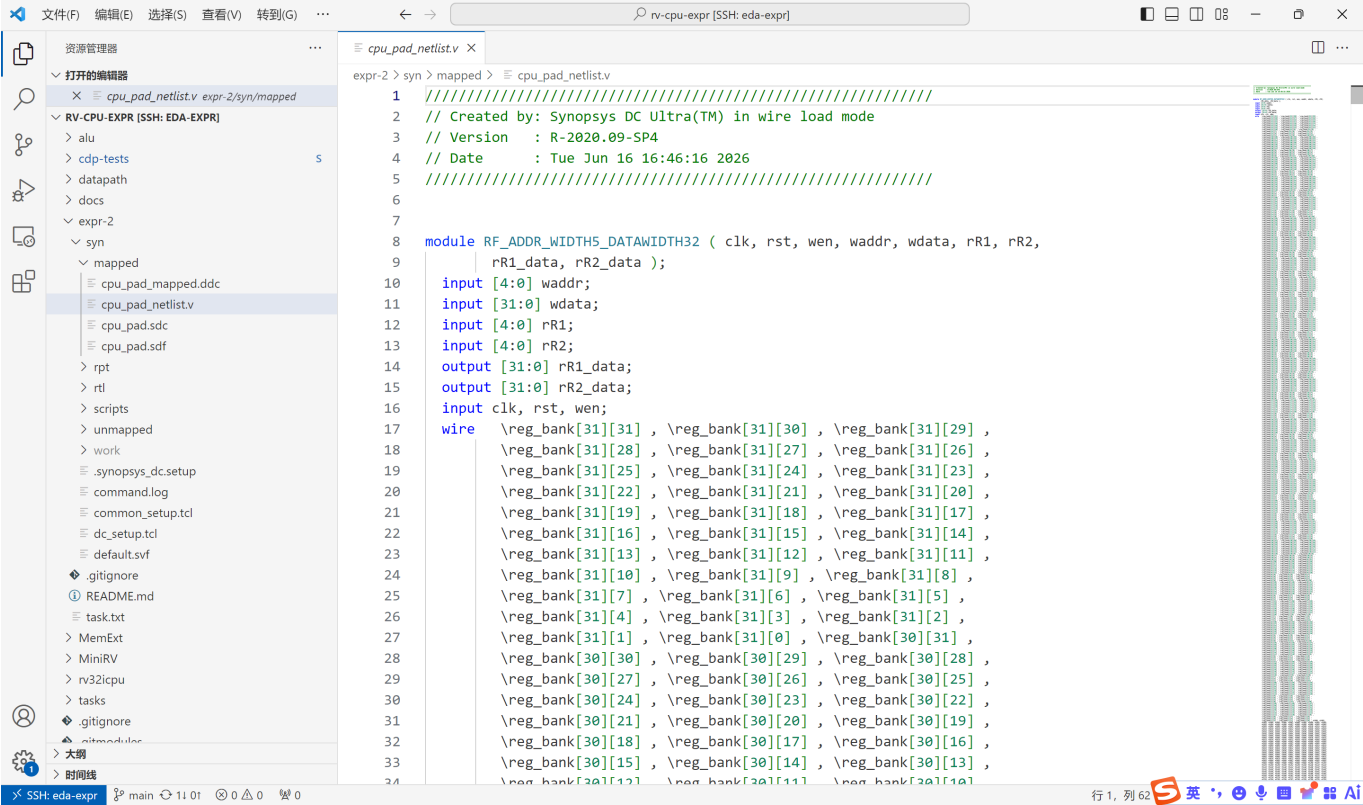
4.6 综合输出文件

文件	用途	下游工具
<code>mapped/cpu_pad_netlist.v</code>	门级网表	ICC2 (布局布线), VCS (后仿)
<code>mapped/cpu_pad.sdc</code>	时序约束	ICC2 (布局布线输入约束)
<code>mapped/cpu_pad.sdf</code>	标准延迟格式	VCS (门级后仿反标)
<code>mapped/cpu_pad_mapped.ddc</code>	DC 二进制数据库	Formality (形式验证)

4.7 遇到的问题

- `target_library` 未自动设置：DC 使用 `target_library` 和 `link_library` 变量（而非 `TARGET_LIBRARY_FILES`），需显式翻译。最初只定义了后者，导致库加载失败。
- `remove_ideal_network` 多端口问题：部分 ICC2 命令不支持一次传入多个端口，改为逐个调用。

4.8 结果截图



五、实验三：版图物理设计

5.1 设计目标

对实验二综合生成的门级网表 (cpu_pad_netlist.v) 和 SDC 约束进行完整的物理设计：布图规划 → 布局 → 时钟树综合 → 布线 → RC 寄生提取 → PT 生成 SDF → VCS 门级后仿真。

5.2 目录结构

expr-3/

rm_setups/

icc_setup.tcl

lcrm_setup.tcl

scripts/

design_setup.tcl

floorplan.tcl

place.tcl

cts.tcl

route.tcl

sdf_gen.tcl

design_data/

run/

output/

工艺库和工具配置

TECH_FILE, MW ref libs, TLU+, design vars

target_library / link_library

各阶段 ICC2/PT 脚本

1. MW 库 + 读网表 + SDC + TLU+

2. Pad/Core + 电源轨

3. 标准单元布局

4. 时钟树综合

5. 布线 + 寄生提取

6. PT SDF 生成

输入网表 + SDC (从 expr-2 复制)

执行目录 + MW 库

最终输出

5.3 物理设计流程

在 ICC2 中通过 Milkyway 库管理设计数据，每个阶段从上一阶段 copy CEL，操作后保存新 CEL：

8 / 14

```
data_setup → floorplan → place → cts → route → PT SDF → VCS 后仿
(42s)      (30s)      (87s)   (73s)   (136s)   (9s)      (2s)
```

阶段 1：数据准备 (`design_setup.tcl`)

1. **创建 MW 物理库**——`create_mw_lib` 使用工艺 TF 文件（8 层金属）和 MW 参考库（标准单元 FRAM + PAD FRAM）
2. **读入门级网表**——`read_verilog` 读取综合网表，`uniquify` 唯一化多实例模块
3. **设置 TLU+ 寄生模型**——StarRC 的互连线 RC 查表模型，分别设置 max/min corners
4. **建立电源/地连接**——`derive_pg_connection` 将全部标准单元的 VDD/VSS pin 连到对应网络
5. **读入 SDC 约束**——复用实验二的时钟定义和输入输出延迟
6. **初始时序检查**——先以零连线延迟模式快速验证，再恢复真实模式

阶段 2：布图规划 (`floorplan.tcl`)

1. **PAD 放置**——创建 corner cell (PCORNER) 和电源 PAD (PVDD1/PVSS1/PVDD2/PVSS2)，ICC2 自动排列所有信号 PAD
2. **Core 创建**——`create_floorplan -core_utilization 0.5 -core_aspect_ratio 1`，方形 core，50% 利用率（234 个 PAD 需要充足空间）
3. **Pad filler 填充**——插入多种尺寸的 filler cell 保证 PAD 环连续性
4. **电源轨布线**——`preroute_standard_cells` 为每行标准单元铺设 VDD/VSS 轨

保存三个中间 CEL：`floorplan_prepn`（电源规划前）→ `floorplanafterpn`（电源连接后）→ `floorplaned`（最终布图规划）。

阶段 3：布局 (`place.tcl`)

1. 设置时钟网为 `ideal_network`（零延迟、无限驱动）
2. `create_fp_placement -timing`——时序驱动的粗放置
3. `legalize_placement`——将单元"卡"到标准单元行上的合法 site，消除重叠

布局阶段不做时序优化。因为此时没有时钟树，高扇出时钟路径的 setup/hold 无法真实评估，psynopt 会在 WNS 极高的情况下无限循环。

阶段 4：时钟树综合 (`cts.tcl`)

1. 移除时钟的 `ideal_network` 属性，让 CTS 接管
2. 设置目标 skew < 0.2ns、目标插入延迟 0.9ns
3. `clock_opt -no_clock_route -only_cts`——插入树形 buffer chain，平衡 1,093 个时钟负载
4. `route_zrt_group -all_clock_nets`——优先对时钟网进行布线

CTS 在时钟源和每个触发器之间插入多级 buffer，将一条驱动 1,000 个负载的网拆分为多级扇出的树形结构。

阶段 5：布线 (`route.tcl`)

1. 再次确认电源/地连接
2. 时钟网优先布线
3. `route_opt -initial_route_only`——信号线初始布线（全局规划 + 轨道分配）

4. `verify_zrt_route + route_zrt_detail`——DRC 检查和修复
5. `extract_rc -coupling_cap`——提取互连线电阻 R + 对地电容 C + 耦合电容 CC

输出：`cpu_pad_final.v` (573 KB) · `cpu_pad.spef.max.gz / .min.gz` (4.6 MB) · `cpu_pad.sdf` (3.8 MB)。

阶段 6：PT SDF 生成 (`sdf_gen.tcl`)

PrimeTime 读取版图后网表和 SPEF 寄生参数，通过 NLDM 非线性延迟模型计算每个门的实际 pin-to-pin 延迟，生成签核级 SDF：

```
read_verilog ../output/cpu_pad_final.v
read_parasitics -pin_cap_included ../output/cpu_pad.spef.max.gz
write_sdf ../output/cpu_pad_pt.sdf
```

输出 `cpu_pad_pt.sdf` (908 KB, 29,582 行) · PT 的延迟模型比 ICC2 自带 `write_sdf` 更精确，是签核 (signoff) 级别的。

阶段 7：VCS 门级后仿真

```
vcs -full64 \
-v /home/eda/houfang/smic13g.v \
-v /home/eda/houfang/SP013D3_V1p2.v \
tb_cpu_pad_post.v cpu_pad_final.v \
+maxdelays -R
```

`tb_cpu_pad_post.v` 例化版图后 `cpu_pad`，提供时钟和复位，通过 `$sdf_annotate()` 反标 PT 生成的 SDF。仿真结果：

```
Doing SDF annotation ..... Done
Post-layout simulation completed.
Debug: have_inst=1 pc=000000c4 ena=1 reg=0 value=00000000
$finish at simulation time 1090000 ps
```

SDF 反标成功，门级仿真正常结束，无 setup/hold 违例报错。

5.4 全流程执行统计

阶段	工具	耗时	内存
data_setup	ICC2	~42s	1,236 MB
floorplan	ICC2	~30s	~1,200 MB
place	ICC2	~87s	1,258 MB
cts	ICC2	~73s	1,337 MB

阶段	工具	耗时	内存
route	ICC2	~136s	1,468 MB
ICC2 合计		~6 min	
PT SDF	PrimeTime	~9s	2,695 MB
VCS 后仿	VCS	~2s	—

5.5 遇到的问题（7 个）

全流程调试中解决了多个 ICC2 工具 bug 和设计问题：

#	问题	工具	原因	解决方案
1	link_library 未设置	ICC2	ICC2 不自动读取 DC 式 .synopsys_dc.setup	在 lcrm_setup.tcl 中显式设 set link_library
2	Power plan 断言崩溃	ICC2	compile_power_plan -ring 触发几何计算 bug (y1>=y2 断言失败)	跳过 ring/mesh，仅用标准单元电源轨
3	place_opt 内部错误	ICC2	can't unset "alo_initial_cluster" 工具变量初始化问题	改用传统 create_fp_placement + legalize_placement
4	Coarse placer 崩溃	ICC2	refine_placement - congestion_effort high 的外部分析进程 (rpsa_exec) 崩溃	移除 congestion-aware placement
5	psynopt 时序优化发散	ICC2	WNS=32ns，没时钟树时无法真正优化高扇出路径	Placement 阶段不做时序优化
6	route_opt 长时间不收敛	ICC2	post-route 全优化在 WNS=14ns 时每次迭代仅改善 ~0.05ns	简化为 initial route + DRC 修复
7	SPEF 标注部分不匹配	PT	40K+ PARA-124/044 错误，SPEF 和网表命名规则不完全一致	不影响 SDF 生成，大部分延迟仍正确标注

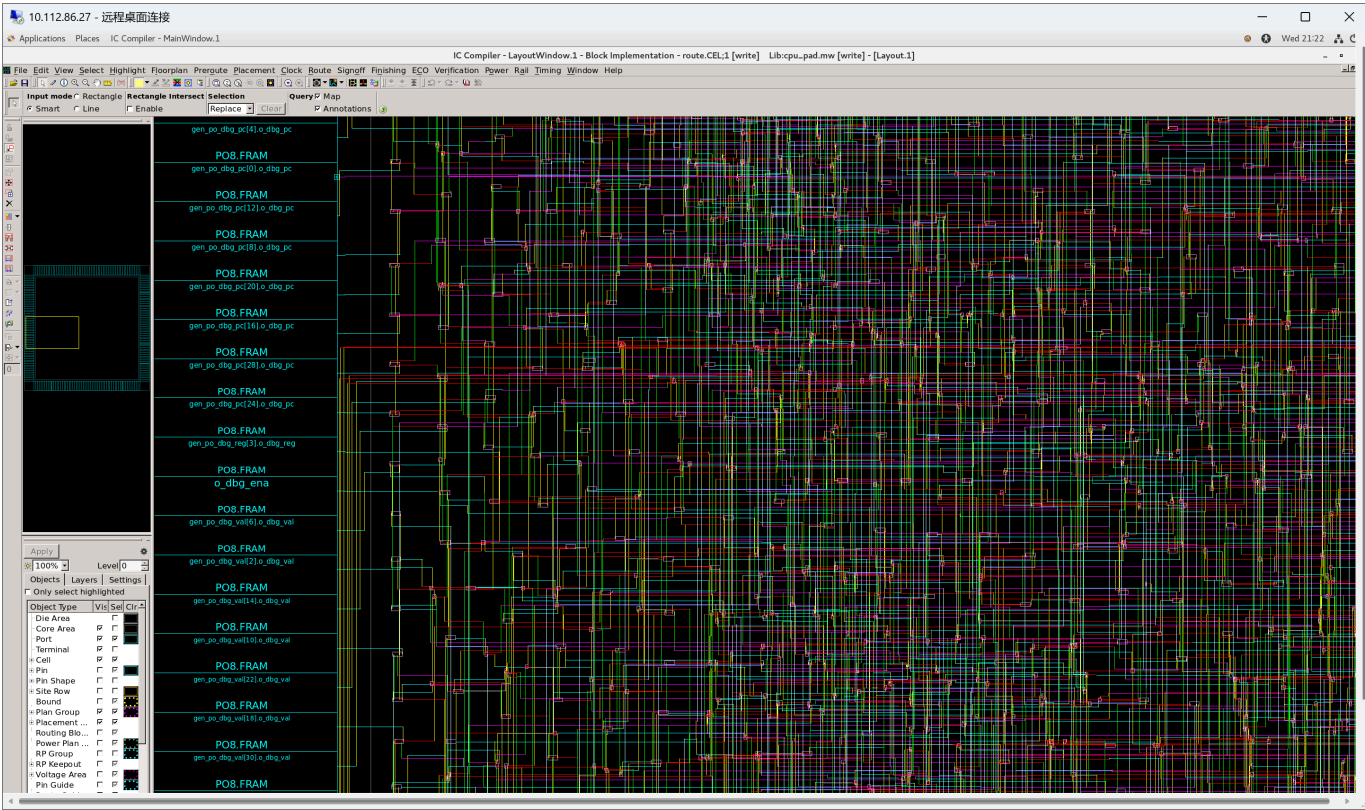
5.6 版图各阶段视图

MW 库保存了每个阶段的版图（CEL），可通过 view_layout.sh 使用 ICC2 GUI 按物理设计流程逐一查看：

顺序	CEL	可观察内容
1	data_setup	刚读入网表，无物理信息
2	floorplan_prepns	PAD 环 + core 边界，电源规划前
3	floorplanafterpn	电源/地连接建立
4	floorplaned	pad filler + 电源轨完成
5	placed	标准单元放置（可见 cell 密度分布）

顺序	CEL	可观察内容
6	cts	时钟树 buffer chain 可见
7	route	最终版图——全部 8 层金属连线可见

5.7 结果截图



六、工具链与自动化

6.1 完整工具链

RTL (SystemVerilog)	Vivado / Verilator (前端仿真)
↓ analyze + elaborate	
↓ compile_ultra	Design Compiler (逻辑综合)
Gate Netlist (.v + .sdc)	
↓ create_mw_lib	
↓ floorplan → place	
↓ cts → route	ICC2 (版图设计)
Layout Netlist + SPEF	
↓ read_parasitics	PrimeTime (时序签核)
SDF (Standard Delay Format)	
↓ \$sdf_annotate	VCS (门级后仿真)
Gate-level Simulation	

6.2 脚本自动化

实验	入口	功能
实验一	<code>cdp-tests/</code> Makefile	Verilator 编译 + 37 项差分测试
实验二	<code>expr-2/syn/scripts/dc_scripts.tcl</code>	DC 综合 + 报告输出
实验三	<code>expr-3/run/run_all.sh</code>	ICC2 五阶段全流程一键运行
实验三	<code>expr-3/run/view_layout.sh</code>	ICC2 GUI 查看各阶段版图

七、设计结果汇总

指标	前端 (RTL)	综合 (DC)	版图 (ICC2)
工艺	—	SMIC 0.13μm	SMIC 0.13μm
目标频率	—	50MHz (20ns)	50MHz (20ns)
单元数	—	5,556	5,556
组合单元	—	4,229	—
时序单元	1,088 bit	1,093	1,093
PAD 数	230 引脚	234	234
等效门数	—	~5,600	—
标准单元面积	—	73,175 μm ²	—
总芯片面积	—	1,686,605 μm ²	—
功耗	—	~252 mW	—
验证结果	37/37 测试通过	Slack=0.00ns	后仿真正常

八、总结

本综合实验完整经历了一款 RISC-V 处理器从 RTL 到版图的数字 IC 设计全流程。

前端设计通过 5 个子实验渐进式构建——从底层 ALU 的标志位计算，到数据通路各部件（PC·RegFile·ImmGen·IM·DM）的时序/组合逻辑分离设计，再到存储器的层次化字位扩展，最后通过控制器集成和指令集扩展完成了支持 37 条指令的 MiniRV 单周期 CPU。使用差分测试框架与大覆盖率指令测试验证了功能正确性。

逻辑综合使用 Design Compiler 将前端 RTL 映射到 SMIC 0.13μm 标准单元库。为 myCPU 添加了 234 个 IO PAD 单元（PI 输入 + PO8 输出），设置了完整的时序约束（时钟 50MHz、输入输出延迟、负载），使用 `compile_ultra` 完成了时序驱动的逻辑优化和面积恢复。综合结果满足 50MHz 目标频率（Slack=0ns），面积约 5,600 等效门。

版图设计使用 ICC2 完成了六阶段物理设计。从 MW 库创建和网表导入开始，经过布图规划（PAD 环 + core + 电源轨）、标准单元布局、时钟树综合（为 1,093 个触发器构建 buffer chain）·信号线布线·寄生参数提取，最后通过 PrimeTime 生成签核级 SDF 并使用 VCS 完成门级后仿真。调试了 7 个 EDA 工具问题，积累了从 DC 到 ICC2 的物理设计实战经验。

通过三个实验的递进式训练，掌握了数字 IC 设计完整工具链（DC → ICC2 → PT → VCS）的使用方法，理解了从前端到后端的全流程和关键技术概念：PPA（Performance, Power, Area）的相互制约、Setup/Hold 时序检查、标准单元库与 Milkyway 数据库、时钟树综合与 skew 控制、寄生提取与 SDF 反标等。这些构成了数字 IC 设计工程师的核心技能栈。